UNIT-1

INTRODUCTION TO PYTHON

1.1. INTRODUCTION:

- Python is a general-purpose interpreted, interactive, object-oriented, and high-level programming language.
- Python is multi-paradigm programming language, which allows user to code in several different programming styles.
- Python supports cross platform development and is available through open source.
- It was initially designed by Guido van Rossum in 1991 and developed by Python Software Foundation.
- It was mainly developed for emphasis on code readability, and its syntax allows programmers to express concepts in fewer lines of code.
- Python is used for:
  - **Web Development:** Python can be used on a server to create web applications
  - **Software Development:** Python can be used alongside software to create workflows
  - **Data Base Systems:** Python can connect to database systems. It can also read and modify files
  - **Mathematics:** Python can be used to handle big data and perform complex mathematics.
  - **System Scripting:** Python can be used for rapid prototyping, or for production-ready software development.

1.2. BRIEF HISTORY OF PYTHON:

- Python was conceptualized by **Guido Van Rossum** in the late **1980s**.
- Rossum published the first version of Python code (0.9.0) in February **1991** at the CWI (Centrum Wiskunde & Informatica) in the Netherlands, Amsterdam.
- Python is derived from **ABC** programming language, which is a general-purpose programming language that had been developed at the CWI.
- Rossum chose the name "**Python**", since he was a big fan of Monty Python's Flying Circus.
- Python is now maintained by a core development team at the institute, although Rossum still holds a vital role in directing its progress.

1.3. WHY PYTHON

Python is popularly used programming language because of the following **features**:

- **Easy-to-learn** − Python has few keywords, simple structure, and a clearly defined syntax. This allows the student to pick up the language quickly.
- **Easy-to-read** − Python code is more clearly defined and visible to the eyes.
- **Easy-to-maintain** − Python's source code is fairly easy-to-maintain.
- **A broad standard library** − Python's bulk of the library is very portable and cross-platform compatible on UNIX, Windows, and Macintosh.
- **Interactive Mode** − Python has support for an interactive mode which allows interactive testing and debugging of snippets of code.

- **Portable** − Python can run on a wide variety of hardware platforms and has the same interface on all platforms.
- **Extendable** − You can add low-level modules to the Python interpreter. These modules enable programmers to add to or customize their tools to be more efficient.
- **Scalable** − Python provides a better structure and support for large programs than shell scripting.
- It supports functional and structured programming methods as well as OOP.
- It can be used as a scripting language or can be compiled to byte-code for building large applications.
- It provides very high-level dynamic data types and supports dynamic type checking.
- It can be easily integrated with C, C++, COM, ActiveX, CORBA, and Java.

1.4. WHEN NOT TO USE PYTHON

Disadvantages of Python are:

1. **Speed:** Python is slower than C or C++. But of course, Python is a high-level language, unlike C or C++ it's not closer to hardware.
2. **Mobile Development:** Python is not a very good language for mobile development. It is seen as a weak language for mobile computing. This is the reason very few mobile applications are built in it like Carbonnelle.
3. **Memory Consumption:** Python is not a good choice for memory intensive tasks. Due to the flexibility of the data-types, Python's memory consumption is also high.
4. **Database Access:** Python has limitations with database access. As compared to the popular technologies like JDBC and ODBC, the Python's database access layer is found to be bit underdeveloped and primitive. However, it cannot be applied in the enterprises that need smooth interaction of complex legacy data.
5. **Runtime Errors:** Python programmers cited several issues with the design of the language. Because the language is dynamically typed, it requires more testing and has errors that only show up at runtime.

1.5. PYTHON VERSIONS

1. **Version 0:**
   - In February 1991, van Rossum published the code (labelled version 0.9.0)
   - Features:
     - classes with inheritance, exception handling, functions, and the core datatypes of list, dict, str
     - a module system borrowed from Modula-3
     - Python's exception model also resembles Modula-3 with the addition of an **else** clause
2. **Version 1:**
   - Python reached version 1.0 in January 1994
   - The major new features included in this release were the functional programming tools lambda, map, filter and reduce
   - By version 1.4, Python had acquired several new features. Notable among these are the Modula-3 inspired keyword arguments and built-in support for complex numbers

3. **Version 2:**
   - Python 2.0 introduced list comprehensions, a feature borrowed from the functional programming languages SETL and Haskell.
   - A major innovation in Python 2.2 was the unification of Python's types (types written in C) and classes (types written in Python) into one hierarchy.
   - Python 2.5 was released on September 2006, and introduced the with statement, which encloses a code block within a context manager
4. **Version 3:**
   - Python 3.0 was released on December 3, 2008
   - Python 3.0 a **new version** of the language that is **incompatible** with the 2.x line of releases.
   - The language is mostly the same, but many details, especially how built-in objects like dictionaries and strings work, have changed considerably, and a lot of deprecated features have finally been removed.
   - Also, the standard library has been reorganized in a few prominent places.
   - Latest version is python 3.7

PYTHON 2 VERSUS PYTHON 3

Python 2	Python 3
It is the more stable and transparent version of the Python programming language.	It is the future of Python designed to address the design flaws in the previous versions.
The print-syntax is treated as a statement rather than a function which requires text to be wrapped in parenthesis.	The print is explicitly treated as a function and replaced by the print( ) function in Python 3 which requires an extra pair of parenthesis.
ASCII string type is used by default to store strings.	Unicode is the implicit string type by default.
It simply returns an integer to the nearest whole number when dividing two integers.	It makes integer division more intuitive by using true division for integers and floats.
xrange function reconstructs the sequence every time.	xrange is replaced by range( ) function in Python 3.

1.6. INSTALLING PYTHON

- Python distribution is available for a wide variety of platforms. You need to download only the binary code applicable for your platform and install Python.
- If the binary code for your platform is not available, you need a C compiler to compile the source code manually. Compiling the source code offers more flexibility in terms of choice of features that you require in your installation.

- **Unix and Linux Installation**
  - Open a Web browser and go to https://www.python.org/downloads/.
  - Follow the link to download zipped source code available for Unix/Linux.
  - Download and extract files.
  - Editing the Modules/Setup file if you want to customize some options.

    run ./configure script
    make
    make install
  - This installs Python at standard location /usr/local/bin and its libraries at /usr/local/lib/pythonXX where XX is the version of Python.
- **Windows Installation**
  - Open a Web browser and go to https://www.python.org/downloads/.
  - Follow the link for the Windows installer python-XYZ.msi file where XYZ is the version you need to install.
  - To use this installer python-XYZ.msi, the Windows system must support Microsoft Installer 2.0. Save the installer file to your local machine and then run it to find out if your machine supports MSI.
  - Run the downloaded file. This brings up the Python install wizard, which is really easy to use. Just accept the default settings, wait until the install is finished, and you are done.
- **Macintosh Installation**
  - Recent Macs come with Python installed, but it may be several years out of date.
  - See http://www.python.org/download/mac/ for instructions on getting the current version along with extra tools to support development on the Mac.
  - For older Mac OS's before Mac OS X 10.3 (released in 2003), MacPython is available.
  - Jack Jansen maintains it and you can have full access to the entire documentation at his website − http://www.cwi.nl/~jack/macpython.html. You can find complete installation details for Mac OS installation.

1.7. ENVIRONMENT VARIABLES

a. **PYTHONPATH:** It has a role similar to PATH. This variable tells the Python interpreter where to locate the module files imported into a program. It should include the Python source library directory and the directories containing Python source code. PYTHONPATH is sometimes preset by the Python installer.

b. **PYTHONSTARTUP:** It contains the path of an initialization file containing Python source code. It is executed every time you start the interpreter. It is named as .pythonrc.py in Unix and it contains commands that load utilities or modify PYTHONPATH.

c. **PYTHONCASEOK:** It is used in Windows to instruct Python to find the first case-insensitive match in an import statement. Set this variable to any value to activate it.

d. **PYTHONHOME:** It is an alternative module search path. It is usually embedded in the PYTHONSTARTUP or PYTHONPATH directories to make switching module libraries easy.

1.8. SETTING UP PATH

- Programs and other executable files can be in many directories, so operating systems provide a search path that lists the directories that the OS searches for executables.

- The path is stored in an environment variable, which is a named string maintained by the operating system. This variable contains information available to the command shell and other programs.
- The path variable is named as PATH in Unix or Path in Windows (Unix is case sensitive; Windows is not).
- In Mac OS, the installer handles the path details. To invoke the Python interpreter from any particular directory, you must add the Python directory to your path.

Setting path at Unix/Linux

- To add the Python directory to the path for a particular session in Unix –
- In the csh shell − type setenv PATH "$PATH:/usr/local/bin/python" and press Enter.
- In the bash shell (Linux) − type export ATH="$PATH:/usr/local/bin/python" and press Enter.
- In the sh or ksh shell − type PATH="$PATH:/usr/local/bin/python" and press Enter.

Note − /user/local/bin/python is the path of the Python directory

Setting path at Windows

- To add the Python directory to the path for a particular session in Windows –
- At the command prompt − type path %path%;C:\Python and press Enter.

Note − C:\Python is the path of the Python directory

1.9. EXECUTING PYTHON FROM THE COMMAND LINE

1.10. IDLE

- IDLE is Python's Integrated Development and Learning Environment.
- **Features:**

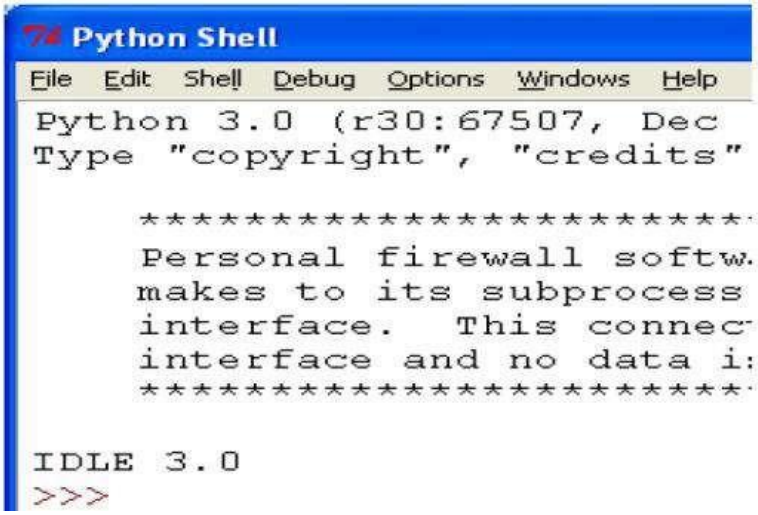
  IDLE has the following features:
  - coded in 100% pure Python, using the tkinter GUI toolkit
  - cross-platform: works mostly the same on Windows, Unix, and Mac OS X
  - Python shell window (interactive interpreter) with colorizing of code input, output, and error messages
  - multi-window text editor with multiple undo, Python colorizing, smart indent, call tips, auto completion, and other features
  - search within any window, replace within editor windows, and search through multiple files (grep)
  - debugger with persistent breakpoints, stepping, and viewing of global and local namespaces
  - configuration, browsers, and other dialogs
  - Automatic Indentation is provided.

- IDLE is a graphical user interface for doing Python development and is a standard and free part of the Python system.
- It is usually referred to as an Integrated Development Environment (IDE).
- One can write the Python code or script using the IDLE which is like an editor.
- This comes along with Python bundle.

- The only thing to do is that we have to launch the IDLE program .
- While the Python shell allows the user to work in interactive mode, the Idle allows to write the code and save it with .py extension.

  Ex: myProg.py.

  This file is known as Python script
- Once the program is saved, it can be executed either by clicking the Run module or F5.
- If there are no errors you observe the results.
- The Screenshot for IDLE will be as shown below. The >>> is the prompt, telling you Idle is waiting for you to type something.



- **Your First Program**
  - To develop the Python program, click on the File and select NewFile.
  - This will open a new text editor where you can write your first program.

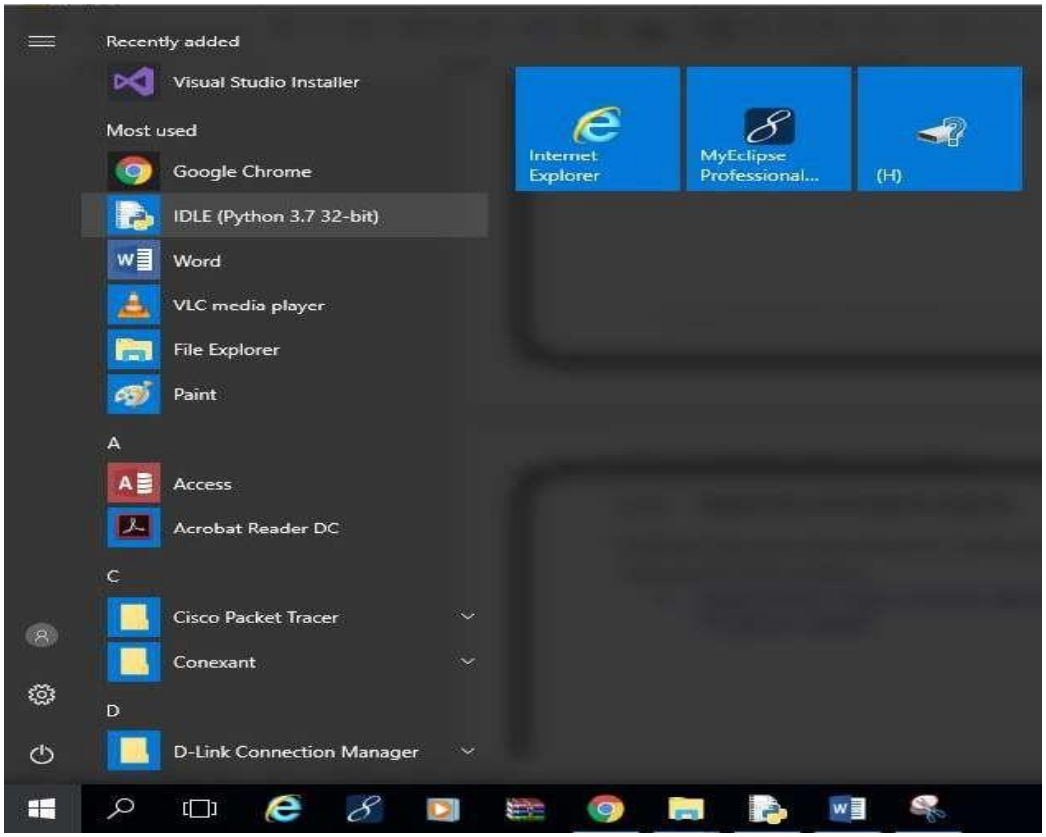
    # Prints the words Hello Python
    print("Hello Python")
    print("Its nice learning Python")
    print("Python is easy to learn")
  - In the above program, the lines appear after the # sign are treated as comments. They are ignored and not executed by Python interpreter. This will help only to improve the readability of the program.
  - After typing the code save this program with an extension .py.
    For Ex: FirstProg.py
  - Now press either F5 or click on Run module.
  - After execution you find the output as

    Hello Python
    Its nice learning Python
    Python is easy to learn.

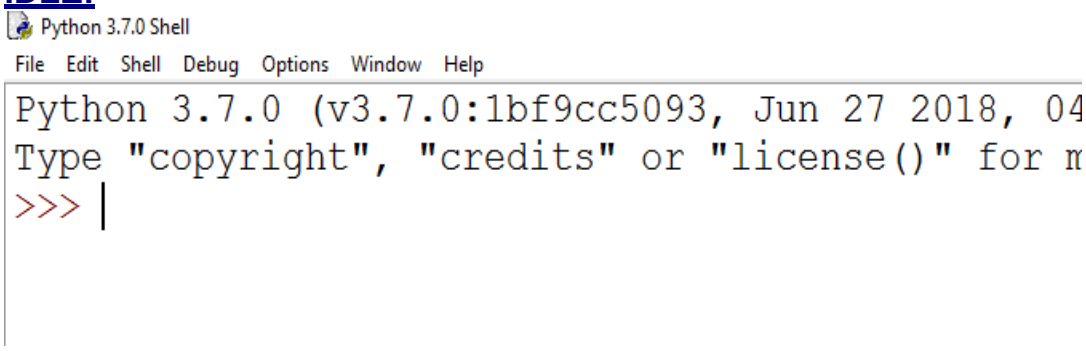
1.11. EDITING PYTHON FILES

- Follow these instructions to write and run a simple Python program using the IDLE editor:

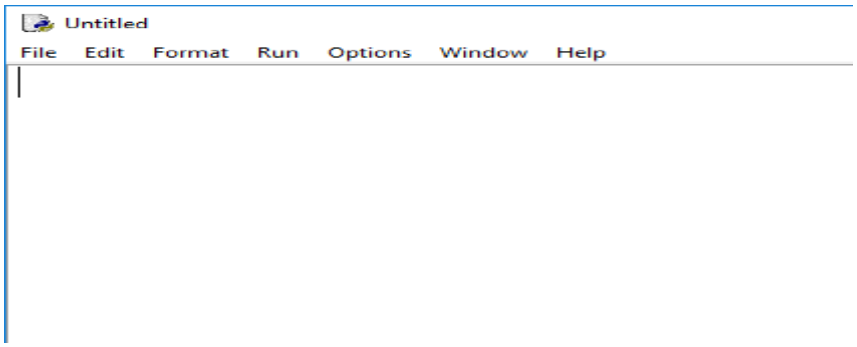
1. Start IDLE (see screen below). You will then see a window entitled "Python Shell"



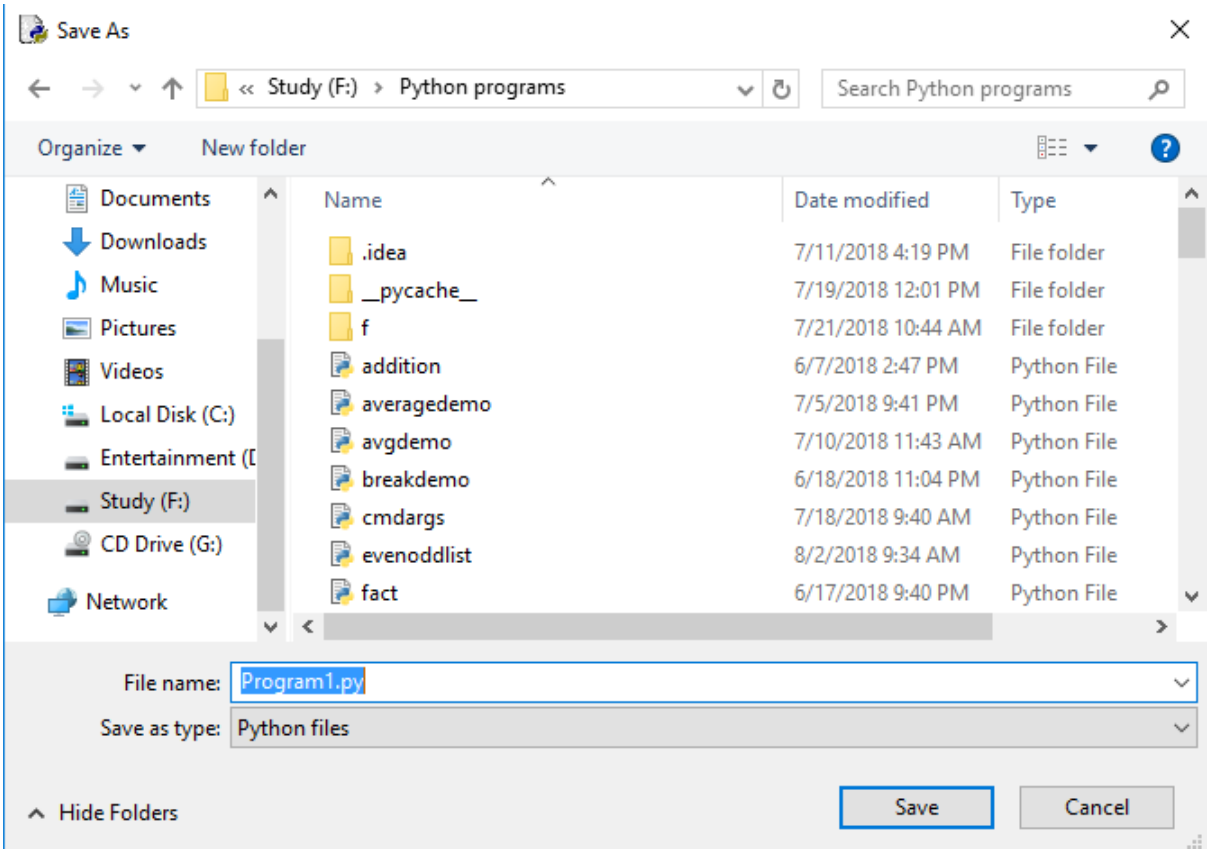
IDLE:



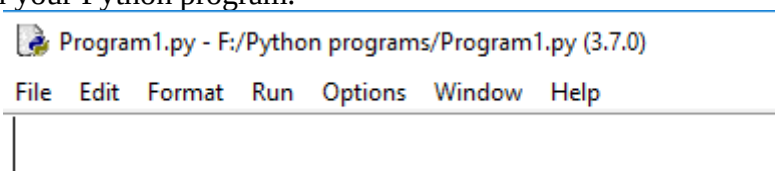
2. From the Python Shell window, select **New Window** from the **File** menu., You will see a window entitled "Untitled"



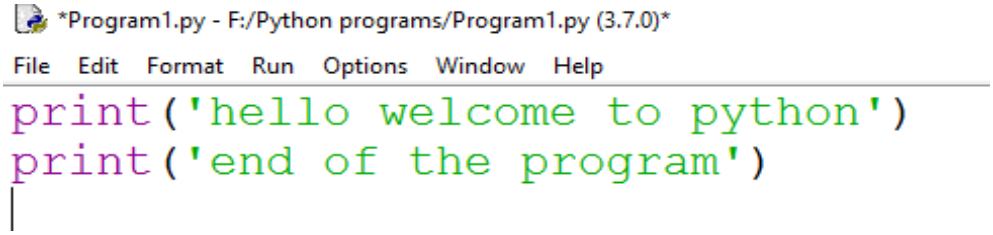
3. From the **File** menu, select **Save As**, and select a folder to save your Python program file.Select a folder to save your file in.In the **File name:** text box, type: `program1.py`

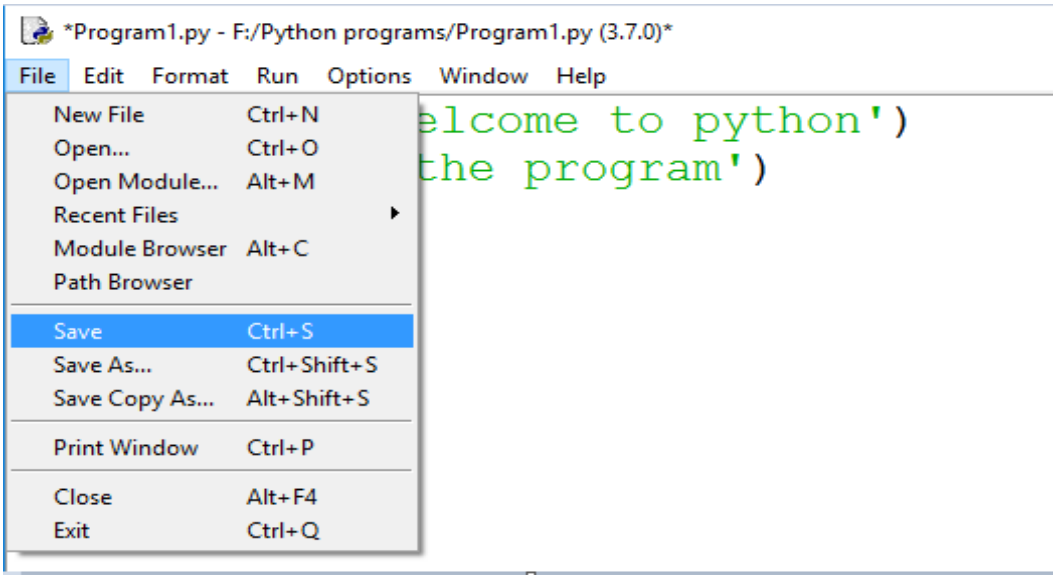


4. Then click on the **Save** button. You will then see a blank editor window ready for you to type in your Python program.

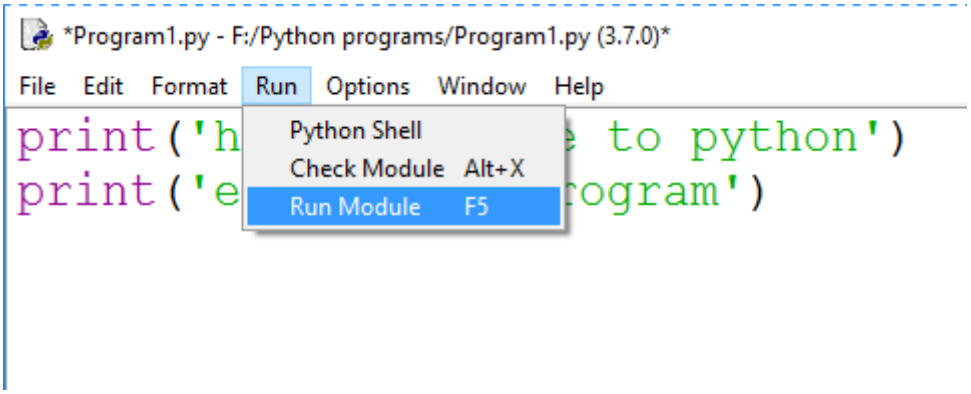


5. Now type the python program, after completion of program goto File→Save Or press Ctrl+S in keyboard.

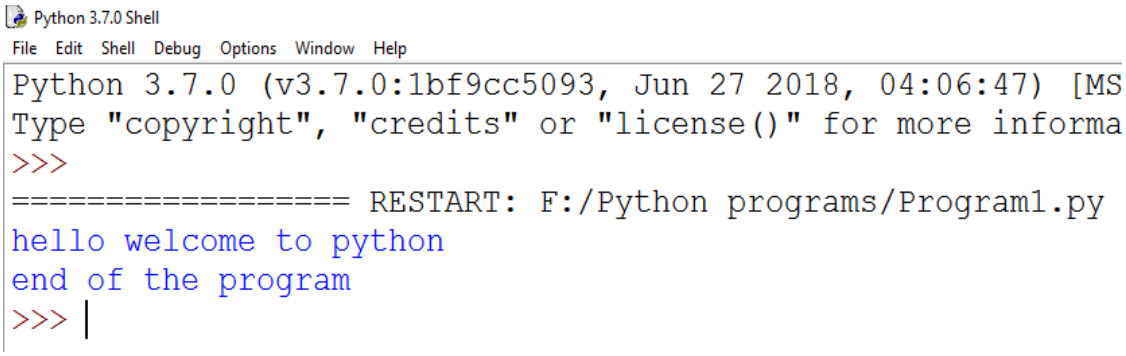




6. After saving the program, to run goto Run→Run Module or Press F5 in keyboard



7. Now the output will be displayed in python shell (IDLE)



1.12. PYTHON RESERVED WORDS

- Python Reserved words are nothing but Keywords
- We cannot use a keyword as variable name, function name or any other identifier. They are used to define the syntax and structure of the Python language.
- In Python, keywords are case sensitive.
- There are 33 keywords in Python 3.3. This number can vary slightly in course of time.
- All the keywords except True, False and None are in lowercase and they must be written as it is.

- The list of all the keywords are given below.

False	class	finally	is	return
None	continue	for	lambda	try
True	def	from	nonlocal	while
and	del	global	not	with
as	elif	if	or	yield
assert	else	import	pass	
break	except	in	raise	

1.13. NAMING CONVENTIONS

What to keep and what to avoid?

- Don't use names that look too general or wordy.
- Maintain a fine line between the two.
- **What is bad?**
  user_list, moveInts, swapNums, dict_to_store_defns_of_a_word
- **What is good?**
  user_info, move_integers, swap_numbers, word_definitions.
- **Things to note-**
  Don't be naive and name things like "X," "Y" or "Z."
  Use CamelCase only when it makes sense.

Package in python?

- Only use lower case while naming a Package in Python.
- Use an underscore as a separator to join multiple words when needed and produce a logical name.
- However, keeping a single word name is what PEP 8 prefer.

Module name?

- Python loves the small case alphabets and hence suggests Module names should also follow the suite.
- Like Packages, combine multiple words with an underscore to generate a meaningful module name.
- Don't think twice if you already have a perfect single word name for your module.

Classe name in Python?

- Classes in Python subject to a different naming scheme called as CapWords convention.
- In some cases, if the class is callable (allows you to use round parenthesis) and has proper documentation, then you can name them like functions.
- Contrary to the above principle, Python has its built-in classes with names in lowercase.
- Exception classes should always have trailing word as "Error."

Global Variables?

- Global variables must all be in lowercase.
- The underscore should be used as a separator while forming a variable with multiple words.

Instance variable name?

- Instance variable names should follow the lowercase convention.
- Have the underscore as a separator while naming a multi-word instance variable.
- Begin a non-public instance variable name with a single underscore. Use two consecutive underscores at the beginning of an instance variable. It will get the name mangled (kind of like a private member).

Method name in Python.

- Make only lowercase Method names.
- An underscore should separate words in a method name.
- The non-public methods should have a leading underscore.
- Mangling a method name will require adding two underscores at the beginning.

Arguments of a method.

- Every instance method should have **'self'** as its first argument.
- All class methods should have **'cls'** as their first argument.

Function name.

- Function names should follow the lowercase naming convention.
- They should also use an underscore to separate words in a function name.

Constants in Python.

- You should be able to identify a Constant just by looking at its name.
- Hence, they all must be in fully capitalized format.
- Words should separate by an underscore to form a long constant name.